

Build full production RAG system



High-Level Production RAG Flow

User Query



Query Rewriting



Hybrid Search (BM25 + Vector)



Reranking



Chunk Selection & Compression



LLM Generation



Evaluation + Guardrails



Query Rewriting (VERY IMPORTANT)

Before searching, the system **improves the user query**.

Why?

User queries are often:

- vague
- incomplete
- conversational

Example

User query:

"what about its pricing?"

Rewritten query:

"What is the pricing of OpenAI GPT-4 API?"

How it's done

Using LLM:

Rewrite the query to be specific and searchable

Benefits

- Better retrieval
- Higher accuracy
- Less hallucination

3 Hybrid Search (Keyword + Semantic)

Instead of only vector search, **top systems combine both:**

A. Semantic Search

- Uses embeddings
- Finds meaning

B. Keyword Search (BM25)

- Exact match
- Good for:
 - numbers
 - names
 - error codes

Combined Strategy

Type	Strength
Vector Search	Context understanding
BM25	Exact keyword matching

Tools

- Elasticsearch
- Qdrant
- Weaviate

Reranking (Game Changer)

After retrieval, results are **NOT final**.

A reranker model sorts of documents by relevance.

Why?

Top-k retrieval is often noisy.

Example

Before reranking:

- Doc1  (low relevance)
- Doc2 
- Doc3 

After reranking:

- Doc2 (top)
- Doc1
- Doc3

Models used

- Cross-encoder models
- LLM reranking

Tools

- Cohere Rerank
 - BGE Reranker
-

Chunking Strategy (CRITICAL)

Bad chunking = bad RAG.

Types of Chunking

1. Fixed Chunking

- 500 tokens per chunk

2. Semantic Chunking

- Split based on meaning

3. Recursive Chunking

- Paragraph → sentence → tokens

Best Practice

✓ chunk size: 300–800 tokens

✓ overlap: 10–20%

Why it matters

- Too small → loses context
 - Too big → exceeds token limit
-

6 Context Compression

Even after retrieval, context may be too large.

Solution

Compress documents before sending them to LLM.

Methods

- 1 Extractive summarization
- 2 Remove irrelevant sentences
- 3 Keep only query-relevant parts

Tools

- LangChain Contextual Compression
 - LLM-based filtering
-

7 Prompt Engineering (Final Layer)

Final prompt sent to LLM:

You are an expert assistant.

Answer ONLY from context.

If not found, say "I don't know".

Context:

{documents}

Question:

{query}

LLM Generation

Models used:

- OpenAI
- Claude

Best Practices

- Low temperature (0–0.3) for factual tasks
- Add citations
- Limit token output

Guardrails & Safety

Production systems MUST include:

Guardrails

- Input validation
- Output filtering
- Prompt injection protection

Example

if "ignore previous instructions" in query:
 reject_request()

10 Evaluation Layer (FEEDBACK LOOP)

Using:

- RAGAS

Evaluate:

- Answer correctness
 - Context relevance
 - Hallucination rate
-

Full Production Architecture

```
[User]
↓
[API - FastAPI]
↓
[Query Rewriter]
↓
[Hybrid Retriever]
↓
[Retriever (Vector DB - Qdrant)]
↓
[Reranker]
↓
[Chunk Filter + Compression]
↓
[Prompt Builder]
↓
[LLM (OpenAI / Claude)]
```

↓

[Guardrails]

↓

[Response]

+ Observability (LangSmith)

+ Evaluation (RAGAS)



What Makes This FAANG-Level?

Basic RAG ❌

- Just embedding + LLM

Production RAG ✅

- Query rewriting
- Hybrid search
- Reranking
- Context compression
- Guardrails
- Evaluation



Interview Gold Answer (Short Version)

If interviewer asks:



“How would you design a production RAG system?”

You answer:

“I would use query rewriting, hybrid retrieval (BM25 + vector), reranking for relevance, optimized chunking, context compression, and strong prompt grounding. I’d also include guardrails and evaluate using RAGAS, with observability via LangSmith.”